RUN MORE BROWSER TASKS · CONTROL WHERE AGENTS CAN GO · REVIEW WHAT THEY ACCESSED

A Secure, Auditable Browser for AI Agents.

- **Run more browser tasks in parallel.** ~5× faster reads with ~80% less peak memory.
- **Start sandboxed by default.** Isolate the browser or the agent's entire workflow inside a container or microVM.
- **Review exactly what happened.** Every browser action and network request is recorded.

★ 580+ GitHub stars · Apache 2.0 · local-first, no hosted service

WHY NOW

Web browsing for agents is too heavy and too risky.

TODAY

Browsers built for people

- ✕ Browsers designed for people are too complex, slow, and memory-intensive
- ✕ Malicious pages can manipulate agents into harmful actions, such as leaking credentials
- ✕ Existing browser tools do not provide a clear, complete audit trail of what the agent did

WITH H5I

A browser built for agents

- ✓ ~5× faster reads with ~80% less peak memory on simple websites
- ✓ Sandboxing and network rules limit what a manipulated agent can access
- ✓ Every browser action and network request is recorded in one reviewable timeline

THE PROBLEM

Browsers built for people are a poor fit for agents.

THE ENGINE

Human features waste compute

Tabs, extensions, device sync, and a full interactive UI consume memory and CPU.

THE OUTPUT

Agents need structured content

Pixels and raw HTML add noise and tokens when the agent needs a compact info.

THE PAGE

Every page is untrusted input

Prompt injections can manipulate an agent into harmful actions.

THE PROFILE

A personal browser carries personal access

Cookies, logged-in accounts, local services, and private networks.

THE RECORD

Automation logs are difficult to audit

Clicks alone do not show every request, blocked connection, or session ending.

APP TESTING

Untrusted code runs directly on your machine

Web Apps might might maliciously behave in the unprotected host.

THE SOLUTION

h5i is built for agent workloads and agent risks.

THE ENGINE

A lightweight engine for agent tasks

~5× faster reads with ~80% less peak memory on simple websites.

THE OUTPUT

Structured pages instead of visual noise

Agents receive compact outlines with `@ref` handles.

THE PAGE

Limit the damage of prompt injection

Page content is marked as untrusted, reducing the risk of manipulated agents.

THE PROFILE

Every session starts with a fresh profile

No inherited cookies or logged-in accounts. Credentials can be brokered.

THE RECORD

Every action and request is reviewable

Allowed and denied requests, crashes, and session endings are recorded.

APP TESTING

Sandbox the entire web workflow

Place the agent, browser, and dependencies inside one sandbox.

THE BROWSER

Run more browser sessions without losing control.

~5× faster reads

~80% less peak memory.

Pages agents can use

Compact snapshots with `@ref` handles.

Every session is reviewable

Actions, requests, denials, and endings.

what the agent receives

```
@e1 heading "Documentation"   @e2 link "Getting started"   @e3 searchbox "Search"
```

THE SANDBOX

Put the agent's entire workflow inside one boundary.

Turn agent permission prompts off: the box is the permission. Writes stay in `$WORK`, egress is allowlisted, every denial is recorded, and an everyday box is ready in under 200 ms.

The agent **still finishes the task**. Only the reviewed patch is merged.

TRACTION

Agent-heavy developers are already pulling this.

580+	50+	20+	20+
GitHub stars	forks	contributors	releases since March

BEACHHEAD

Start with developers already giving agents browser access.

Developers already use agents to browse and test web apps. Now they need a safer, lighter way to run more sessions.

BEACHHEAD · BUYS FIRST

Developers building browser-enabled agents

Need fast local browsing, structured page output, safe web app testing, and records they can review.

EXPANSION →

Agent platforms

Run many concurrent browser sessions with lower memory use.

Platform and DevEx teams

Provide one isolated browser and testing workflow across developer laptops.

Security teams

Restrict browser access, keep credentials outside the agent.

Based on dozens of developer interviews during PhD research in AI and systems security.

WHY H5I WINS

Other browsers solve speed or security. h5i solves both.

Conventional browser automation	Built for people, so it is slow, memory-heavy, difficult to audit, and exposes more access than agents need.
Lightweight browsers	Reduce startup time and memory, but do not limit what a manipulated agent can access or send.
Sandboxing an existing browser	Reduces risk, but keeps the cost and complexity of a browser built for people.
h5i	**~5× faster reads and ~80% less peak memory on simple websites, with request records, network controls, and sandboxing built in.**

Fast enough to run more sessions. Controlled enough to grant more autonomy.

WHY NOW · WHY US

We are making h5i the default browser for AI agents.

Why now

- Agents are increasingly browsing websites and testing web apps with less human supervision.
- Existing browsers make teams choose between performance and control.

Why us

- 500+ GitHub stars, 20+ contributors, and third-party tools built by users.
- Grounded in PhD research and dozens of developer interviews on agent security.

Next 12 weeks

- Expand website compatibility and ship the read-only dashboard.
- Convert 10 design partners and land the first paid pilots.

Give your agent **a browser**. **Just not yours.**

h5i.dev

github.com/h5i-dev/h5i